

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Самсонов И.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 01.10.26

Москва, 2026

Постановка задачи

Вариант 18.

Родительский процесс создаёт два дочерних процесса. Первые две строки ввода задают имена файлов для результатов первого и второго дочерних процессов. Нечётные строки родитель направляет первому процессу через `pipe1`, чётные — второму процессу через `pipe2`. Дочерние процессы удаляют из строк гласные латинские буквы и записывают результат в назначенные файлы.

Общий метод и алгоритм решения

Решение состоит из двух программ: `parent` и `child`. Родитель читает имена выходных файлов, создаёт два неименованных канала и два дочерних процесса. Каждый потомок перенаправляет стандартный ввод на читающий конец своего канала, стандартный вывод — в соответствующий файл, после чего запускает программу `child`.

Использованные системные вызовы:

- `pipe` — создание однонаправленных каналов;
- `fork` — создание дочерних процессов;
- `open` — создание и открытие выходных файлов;
- `dup2` — перенаправление стандартных потоков;
- `execl` — замена образа дочернего процесса программой `child`;
- `read` и `write` — чтение и передача данных;
- `close` — закрытие неиспользуемых файловых дескрипторов;
- `waitpid` — ожидание завершения дочерних процессов;
- `sigaction` — настройка обработки `SIGPIPE`;
- `_exit` — завершение дочернего процесса при ошибке.

Родитель считывает пользовательский ввод посимвольно. Номер текущей строки начинается с единицы и увеличивается после символа перевода строки. Для нечётного номера символ записывается в `pipe1`, для чётного — в `pipe2`. Такой способ не ограничивает длину обрабатываемой строки размером буфера.

Программа `child` читает свой стандартный ввод до EOF, пропускает символы A, E, I, O, U в обоих регистрах и записывает остальные символы в стандартный вывод. После закрытия родителем записывающих концов каналов дочерние процессы получают EOF, завершаются, а родитель проверяет их статусы. Ошибки системных вызовов обрабатываются; вызовы, прерванные сигналом `EINTR`, повторяются.

Код программы

parent.c

```
#include <signal.h>
#include <errno.h>
#include <fcntl.h>
#include <stddef.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#define FILE_NAME_SIZE 256

int wait_for_child(pid_t child){
    int status;
    pid_t result;

    do {
        result = waitpid(child, &status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
```

```

    return -1;
}

if (!WIFEXITED(status)){
    return -1;
}
if (WEXITSTATUS(status) != 0){
    return -1;
}
return 0;
}

void print_error(const char *message, size_t size){
    write(STDERR_FILENO, message, size);
}

int read_file_name(char *file_name){
    int index = 0;
    char character;

    while (index < FILE_NAME_SIZE - 1){
        ssize_t bytes_read = read(STDIN_FILENO, &character, 1);

        if (bytes_read == -1){
            if (errno == EINTR){
                continue;
            }
            return -1;
        }
        if (bytes_read == 0 || character == '\n'){
            break;
        }
        file_name[index] = character;
        index++;
    }

    file_name[index] = '\0';

    if (index == 0){
        return -1;
    }
    return 0;
}

int main(void)
{
    struct sigaction action;
    action.sa_handler = SIG_IGN;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;
    if (sigaction(SIGPIPE, &action, NULL) == -1){
        const char message[] = "Error: cannot configure SIGPIPE\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    char first_file_name[FILE_NAME_SIZE];
    char second_file_name[FILE_NAME_SIZE];

    const char first_prompt[] = "First file name: ";
    write(STDOUT_FILENO, first_prompt, sizeof(first_prompt) - 1);

    if (read_file_name(first_file_name) == -1){
        const char message[] = "Error: invalid first file name\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    const char second_prompt[] = "Second file name: ";
    write(STDOUT_FILENO, second_prompt, sizeof(second_prompt) - 1);

    if (read_file_name(second_file_name) == -1){
        const char message[] = "Error: invalid second file name\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    int pipe2[2];
    int pipe1[2];

    if (pipe(pipe1) == -1) {
        const char message[] = "Error: cannot create pipe1\n";
        print_error(message, sizeof(message) - 1);
        return 1;
    }

    if (pipe(pipe2) == -1) {
        const char message[] = "Error: cannot create pipe2\n";
        print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(pipe1[1]);
        return 1;
    }

    pid_t child1 = fork();

    if (child1 == -1){
        const char message[] = "Error: cannot create child1\n";
        print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);

```

```

    close(pipe2[1]);
    return 1;
}

if (child1 == 0){
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    int output_file = open(first_file_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_file == -1){
        const char message[] = "Error: child1 cannot open output file\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[0]);
        _exit(1);
    }

    if (dup2(pipe1[0], STDIN_FILENO) == -1) {
        const char message[] = "Error: child1 cannot redirect stdin\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[0]);
        close(output_file);
        _exit(1);
    }

    if (dup2(output_file, STDOUT_FILENO) == -1){
        const char message[] = "Error: child1 cannot redirect stdout\n";
        print_error(message, sizeof(message) - 1);

        close(pipe1[0]);
        close(output_file);
        _exit(1);
    }
    close(pipe1[0]);
    close(output_file);
    execl("./child", "child", (char *)0);
    const char message[] = "Error: cannot execute child\n";
    print_error(message, sizeof(message) - 1);
    _exit(1);
}

pid_t child2 = fork();

if (child2 == -1){
    const char message[] = "Error: cannot create child2\n";
    print_error(message, sizeof(message) - 1);

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(child1, NULL, 0);
    return 1;
}

if (child2 == 0){
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);

    int output_file = open(second_file_name, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_file == -1){
        const char message[] = "Error: child2 cannot open output file\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        _exit(1);
    }

    if (dup2(pipe2[0], STDIN_FILENO) == -1){
        const char message[] = "Error: child2 cannot redirect stdin\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        close(output_file);
        _exit(1);
    }

    if (dup2(output_file, STDOUT_FILENO) == -1){
        const char message[] = "Error: child2 cannot redirect stdout\n";
        print_error(message, sizeof(message) - 1);
        close(pipe2[0]);
        close(output_file);
        _exit(1);
    }

    close(pipe2[0]);
    close(output_file);
    execl("./child", "child", (char *)0);
    const char message[] = "Error: cannot execute child\n";
    print_error(message, sizeof(message) - 1);
    _exit(1);
}

close(pipe1[0]);
close(pipe2[0]);

char character;
int line_number = 1;

while (1){
    ssize_t bytes_read = read(STDIN_FILENO, &character, 1);

```

```

    if (bytes_read == -1){
        if (errno == EINTR){
            continue;
        }
        const char message[] = "Error: cannot read input\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[1]);
        close(pipe2[1]);

        waitpid(child1, NULL, 0);
        waitpid(child2, NULL, 0);
        return 1;
    }
    if (bytes_read == 0){
        break;
    }
    int destination;

    if (line_number % 2 == 1){
        destination = pipe1[1];
    } else{
        destination = pipe2[1];
    }
    ssize_t bytes_written;
    do {
        bytes_written = write(destination, &character, 1);
    } while (bytes_written == -1 && errno == EINTR);

    if (bytes_written != 1) {

        const char message[] = "Error: cannot write to pipe\n";
        print_error(message, sizeof(message) - 1);
        close(pipe1[1]);
        close(pipe2[1]);

        waitpid(child1, NULL, 0);
        waitpid(child2, NULL, 0);
        return 1;
    }
    if (character == '\n'){
        line_number++;
    }
}

close(pipe1[1]);
close(pipe2[1]);

int child1_result = wait_for_child(child1);
int child2_result = wait_for_child(child2);

if (child1_result == -1 || child2_result == -1){
    const char message[] = "Error: child process failed\n";
    print_error(message, sizeof(message) - 1);
    return 1;
}

return 0;
}

```

child.c

```

#include <errno.h>
#include <unistd.h>

int is_vowel(char character){
    return character == 'a' ||
        character == 'e' ||
        character == 'i' ||
        character == 'o' ||
        character == 'u' ||
        character == 'A' ||
        character == 'E' ||
        character == 'I' ||
        character == 'O' ||
        character == 'U';
}

int main(void){
    char character;

    while(1){
        ssize_t bytes_read = read(STDIN_FILENO, &character, 1);

        if (bytes_read == -1){
            if (errno == EINTR){
                continue;
            }
        }

        const char message[] = "Error: child cannot read\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        return 1;
    }

    if (bytes_read == 0){
        break;
    }
    if (is_vowel(character)){
        continue;
    }

    ssize_t bytes_written;
    do {
        bytes_written = write(STDOUT_FILENO, &character, 1);
    } while(
        bytes_written == -1 && errno == EINTR);

    if (bytes_written != 1) {

```

```

    const char message[] = "Error: child cannot write\n";
    write(STDERR_FILENO, message, sizeof(message) - 1);
    return 1;
}
}
return 0;
}

```

Протокол работы программы

Программа собрана компилятором cc с включёнными предупреждениями. Для проверки введены четыре строки: нечётные должны попасть в odd.txt, чётные — в even.txt.

Тестирование:

```

$ cc -std=c11 -Wall -Wextra -Wpedantic -Werror child.c -o child
$ cc -std=c11 -Wall -Wextra -Wpedantic -Werror parent.c -o parent
$ ./parent
First file name: odd.txt
Second file name: even.txt
Operating Systems
Queue audio
Process communication
Fork and pipe
<Ctrl-D>
$ cat odd.txt
prtnng Systms
Prccss cmmnctn
$ cat even.txt
Q d
Frk nd pp

```

Вывод

В ходе лабораторной работы была реализована передача строк между тремя процессами через два неименованных канала. Родительский процесс корректно распределяет строки по чётности их номера, а дочерние процессы удаляют гласные и записывают результаты в разные файлы. Основная сложность состояла в правильном закрытии неиспользуемых концов каналов и перенаправлении дескрипторов через dup2. Обработка EOF, EINTR, SIGPIPE и статусов waitpid обеспечивает корректное завершение программы и диагностику ошибок.